

PRÁCTICA N.º 4 — HOJA DE EVIDENCIAS

GraphQL y el patrón Backend for Frontend

COM-600 · Microservicios · Gestión 2/2026 · Tema 4 — Comunicación entre Microservicios

Nombre y apellidos	
Código universitario (CU)	
Carrera y paralelo	
Fecha de entrega	
Sistema operativo, versión de Node y de MySQL	
Repositorio (URL)	

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Entorno y proyecto base	1, 2	1	
Laboratorio 1 — El esquema como contrato	3, 4	2	
Laboratorio 2 — Resolvers: de dónde salen los datos	5, 6	3	
Laboratorio 3 — Mutaciones: cambiar el estado y devolver algo útil	7, 8	4	
Laboratorio 4 — Composición: aquí aparece el Backend for Frontend	9, 10, 11	5	
Laboratorio 5 — El problema N+1 y la carga por lotes	12, 13	6	
Laboratorio 6 — Errores, límites y seguridad básica	14, 15	7	
Laboratorio 7 — Comparar con REST midiendo, no opinando	16, 17, 18, 19	8	

PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — Entorno y proyecto base

Captura N.º 1: salida de los cuatro comandos anteriores, con el JSON de usuarios visible.

[Pegue aquí la captura de pantalla]

Pregunta N.º 1. El servicio REST de la Práctica 3 sigue funcionando exactamente igual que antes y esta práctica no le cambia una sola línea. ¿Qué gana un equipo cuando la capa nueva se pone delante en lugar de reescribir el servicio existente?

Captura N.º 2: consola de MySQL mostrando el resultado de `SELECT * FROM ventas`; con las tres filas insertadas.

[Pegue aquí la captura de pantalla]

Laboratorio 1 — El esquema como contrato

Captura N.º 3: panel de documentación del explorador mostrando el tipo Venta con sus campos y sus obligaciones.

[Pegue aquí la captura de pantalla]

Captura N.º 4: las dos ejecuciones: la consulta válida con su respuesta y el error Cannot query field al pedir un campo inexistente.

[Pegue aquí la captura de pantalla]

Pregunta N.º 2. En REST, pedirle a un endpoint un campo que no existe suele devolver un 200 con ese campo ausente, y el error aparece recién en el cliente. Aquí el servidor lo rechaza antes de ejecutar nada. ¿Qué implica esa diferencia para un equipo con varios consumidores del mismo servicio?

Laboratorio 2 — Resolvers: de dónde salen los datos

Captura N.º 5: la consulta con argumento y su respuesta anidada en el explorador, junto a las sentencias SQL en la terminal del servidor.

[Pegue aquí la captura de pantalla]

Captura N.º 6: la consulta con variables ejecutada dos veces: con id 2 devolviendo datos y con id 99 devolviendo nulo.

[Pegue aquí la captura de pantalla]

Pregunta N.º 3. Con variables, la consulta enviada al servidor es siempre la misma cadena de texto y solo cambia el JSON de variables. Además del orden, ¿qué otro problema evita esa separación cuando el valor viene escrito por un usuario?

Laboratorio 3 — Mutaciones: cambiar el estado y devolver algo útil

Captura N.º 7: la mutación crearVenta ejecutada, con la respuesta que trae el identificador nuevo, el total calculado y el detalle insertado.

[Pegue aquí la captura de pantalla]

Captura N.º 8: la mutación rechazada, mostrando el arreglo errors con el mensaje «La cantidad debe ser mayor que cero».

[Pegue aquí la captura de pantalla]

Pregunta N.º 4. Compare esta respuesta con la que daría su servicio REST de la Práctica 3 ante la misma situación. ¿Qué pierde un cliente que solo mira el código de estado HTTP cuando habla con un servidor GraphQL?

Laboratorio 4 — Composición: aquí aparece el Backend for Frontend

Captura N.º 9: las dos respuestas del servicio REST de la Práctica 3, con los usuarios 1 y 2.

[Pegue aquí la captura de pantalla]

Captura N.º 10: la consulta compuesta y su respuesta, con el nombre del cliente traído del servicio REST dentro del mismo objeto venta.

[Pegue aquí la captura de pantalla]

Captura N.º 11: la respuesta parcial: data con los campos que sí se resolvieron y errors indicando el fallo del campo cliente.

[Pegue aquí la captura de pantalla]

Pregunta N.º 5. Su capa GraphQL ahora depende de que el servicio REST esté vivo. Nombre dos consecuencias concretas de esa dependencia y una medida que usted tomaría para que la caída del servicio de usuarios no arrastre a toda la pantalla.

Laboratorio 5 — El problema N+1 y la carga por lotes

Captura N.º 12: terminal del servidor mostrando la sentencia de ventas seguida de una sentencia de detalle_venta por cada venta.

[Pegue aquí la captura de pantalla]

Pregunta N.º 6. La consulta responde bien y ningún registro de errores se queja. ¿Por qué este problema suele aparecer recién en producción y no en la máquina del desarrollador?

Captura N.º 13: los dos registros del servidor lado a lado: el de antes, con una sentencia por cada venta, y el de después, con una única sentencia WHERE venta_id IN (...), junto a los dos conteos.

[Pegue aquí la captura de pantalla]

Laboratorio 6 — Errores, límites y seguridad básica

Captura N.º 14: la respuesta del explorador con el error resumido y, al lado, la terminal del servidor con el detalle completo.

[Pegue aquí la captura de pantalla]

Captura N.º 15: el error de profundidad excedida devuelto por el servidor, sin que aparezca ninguna sentencia SQL en la terminal.

[Pegue aquí la captura de pantalla]

Pregunta N.º 7. La profundidad no es la única medida del costo de una consulta: pedir diez mil ventas con dos niveles cuesta más que pedir una venta con seis. ¿Qué otro límite agregaría usted y sobre qué lo calcularía?

Laboratorio 7 — Comparar con REST midiendo, no opinando

Captura N.º 16: las tres mediciones con curl del camino REST, con sus tamaños y tiempos.

[Pegue aquí la captura de pantalla]

Captura N.º 17: la medición con curl de la consulta GraphQL equivalente, con su tamaño y su tiempo.

[Pegue aquí la captura de pantalla]

Captura N.º 18: la tabla de comparación completada con sus propias mediciones.

[Pegue aquí la captura de pantalla]

Pregunta N.º 8. Con sus números a la vista: ¿en qué escenario concreto de su proyecto integrador seguiría prefiriendo REST antes que GraphQL? Justifique con al menos una de las medidas que acaba de tomar.

Captura N.º 19: la página del repositorio en GitHub mostrando la carpeta practica4-graphql con src/, db/ y el README.

[Pegue aquí la captura de pantalla]

Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

¿Qué haría distinto si tuviera que repetir esta práctica?

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

Firma del estudiante

Fecha